

Team Structure & Ownership

Who owns what, and how the five roles hand off to each other.

Read this if: everyone — especially useful for both frontend members

Five people, five lanes. The goal isn't to keep everyone siloed — it's to make sure two people never rebuild the same thing, or block each other, without knowing it.

MEMBER 1

Team Lead — Backend, API, Integration

Owns the FastAPI app structure, API routes/schemas, auth integration, database configuration, the API contract, integrating every backend module, connecting frontend to backend, background jobs, error handling, deployment, end-to-end testing, and architecture/scope decisions. Owns the interfaces and integration — not every feature underneath them.

```
backend/app/main.py · api/ · core/ · schemas/ · models/ · services/integration/

backend/app/graph/ belongs to Member 3 — M1 integrates against its interface.
```

MEMBER 2

Repository Acquisition & Git History Intelligence

Owns the data foundation: validating/cloning public repos, a safe temp workspace, file trees, language detection, commits, diffs, contributors, branches/tags, PRs, issues, and history timelines, plus cleanup. This output needs to be stable — Members 1 and 3 build directly on it.

```
backend/app/services/repository/ · services/history/
```

MEMBER 3

Static Code Intelligence & AI / Knowledge Layer

Static code intelligence: language-specific parsing, code entities, imports, dependencies, architecture signals. `services/parsing/`

AI & LangGraph: the LangGraph workflow, Groq integration, the Historian / Developer Guidance / Knowledge Gap agents, retrieval + embeddings (pgvector), evidence assembly, confidence classification, knowledge storage, and guardrails. `agents/` , `services/retrieval/` , `services/knowledge/` , `services/llm/` , `graph/`

Risk & offboarding: deterministic risk scoring (change frequency, contributor count, ownership concentration, reverts, activity, dependency centrality, knowledge gaps) and the offboarding workflow. `services/risk/` , `agents/offboarding/`

Doesn't touch frontend implementation.

MEMBER 4

Frontend A — Application Shell & Repository Workspace

Owns project setup, routing, auth screens, navigation/layout, dashboard, repository import, repo list/detail views, analysis progress, the file explorer, Monaco integration, the API client foundation, shared types, and loading/error states.

```
pages/ · components/layout/ · features/auth/ · features/repositories/ · features/explorer/ · services/ · types/
```

MEMBER 5

Frontend B — Intelligence & Knowledge Experience

Owns architecture visualization (React Flow), history/timeline UI, the Historian question interface, evidence/confidence display, Developer Guidance UI, the Risk dashboard, Knowledge Gap UI and human-answer form, knowledge reuse UI, the Offboarding workspace/report, the guardrail refusal UI, and final visual polish.

```
features/architecture/ · features/history/ · features/historian/ · features/guidance/ · features/risk/ · features/knowledge/ · features/offboarding/
```

At a glance

Area	Owner	Depends on
FastAPI / API	M1	All
Auth	M1	M4
DB integration	M1	M2 / M3
Repository acquisition	M2	M1
Git history	M2	M3 / M1

Static parsing	M3	M2
Historian / Dev Guidance	M3	M2
Knowledge Gap	M3	M3 knowledge store
Risk / Offboarding	M3	M2 + M3
Frontend shell / explorer	M4	M1
Frontend intelligence views	M5	M1 + M3
Final integration / deploy	M1	All

Shared identifiers

Build against: `user_id`, `repository_id`, `analysis_job_id`, `file_id/path`, `evidence_id`, `knowledge_item_id`, `knowledge_gap_id`, `offboarding_session_id`.

M2 (Repository/History) → M3 (Parsing + AI/Risk/Offboarding) → M1 (API integration) → M4/M5 (Frontend) → E2E testing → Deployment